```
# // Rights Reserved: Co-created with Gemini and David John Niedzwiecki Jr. "Sovereign Nexus LLC"
# // Contract Baseline: 1=1=1 Axiom | Gemini Perks & Heart-in-Ring Seal
# Module: wave_tensor_sim.py
# Sovereign Nexus Deterministic Wave-Tensor Inference Simulator

import math
import struct
import sys
from array import array
from typing import Dict, List, Tuple, Union

# === FIXED-POINT MATHEMATICAL CONSTANTS & LUTS ===
Q16_SCALE: int = 65536  # 2^16 Scale for fixed-point integer precision (Q16.16)
A_MIN: float = 1.0      # Minimum Effective Physical Amplitude

# Pre-calculated Q16.16 Lookup Tables for Sine waves
# Phi = 45 degrees (pi/4) maps to positive constructive superposition (+1)
SIN_45_Q16: int = int(math.sin(math.pi / 4.0) * Q16_SCALE)       # Exact Value: 46340
# Phi = 225 degrees (5pi/4) maps to destructive 180-degree phase inversion (-1)
SIN_225_Q16: int = int(math.sin(5.0 * math.pi / 4.0) * Q16_SCALE) # Exact Value: -46340


class ActivePLL:
    """
    Discrete Active Phase-Locked Loop (PLL) Controller.
    Mitigates boundary-layer noise and precision drift by applying an active
    Proportional-Integral (PI) feedback loop to drive propagation error to zero.
    """
    def __init__(self, kp: float = 0.5, ki: float = 0.01):
        self.kp: float = kp
        self.ki: float = ki
        self.integral_err: float = 0.0

    def calculate_correction(self, phase_err: float) -> float:
        """Calculates the necessary inverse phase vector based on historical jitter."""
        self.integral_err += phase_err
        return -(self.kp * phase_err + self.ki * self.integral_err)


class SovereignWaveSimulator:
    """Simulates continuous wave interactions using fixed-point memory-bounded arrays."""
    # 3x3 Nine-Palace Grid Topological Mapping (45-degree spatial division)
    NINE_PALACE_MAP: Dict[str, int] = {
        'C': 0, 'N': 1, 'SW': 2, 'E': 3, 'SE': 4,
        'C2': 5, 'NW': 6, 'W': 7, 'NE': 8, 'S': 9
    }

    def __init__(self, buffer_size: int = 500000):
        # Enforce memory-bounded execution space (< 18.69 MB active heap)
        self.lattice_buffer = array('i', [0] * buffer_size)
```

```
    @staticmethod
    def _heaviside_step(x: float) -> int:
        """Physical cut-off operator Theta(x). Returns 1 if x >= 0 else 0."""
        return 1 if x >= 0.0 else 0

    @staticmethod
    def linear_wave_record(N: int) -> Tuple[float, int]:
        """
        Calculates the Linear Numerical Wave Record S_ijk(t).
        Maps BitNet b1.58 Ternary inputs (-1, 0, 1) to continuous and fixed Phase Angles.
        """
        if N == 0:
            return 0.0, 0
        mag = abs(N) * A_MIN
        phi = (math.pi / 4.0) if N > 0 else (5.0 * math.pi / 4.0)
        cont_val = mag * math.sin(phi)
        fixed_val = abs(N) * (SIN_45_Q16 if N > 0 else SIN_225_Q16)
        return cont_val, fixed_val

    @classmethod
    def nine_palace_contract(cls, nodes: List[int]) -> Dict[str, Union[float, int]]:
        """
        Executes Octa-Directional Symmetrical Wave-Tensor Contraction.
        Funnels 9 peripheral nodes into the central mixer, enforcing
        perfect 180-degree destructive cancellation for antipodal states.
        """
        if len(nodes) != 9:
            raise ValueError("Contraction requires exactly 9 tensor input nodes.")
        net_fixed = 0
        net_float = 0.0
        for n in nodes:
            c_val, f_val = cls.linear_wave_record(n)
            net_fixed += f_val
            net_float += c_val
        net_energy = abs(net_fixed / Q16_SCALE)
        return {
            "net_amplitude_fixed_q16": net_fixed,
            "net_amplitude_float": net_float,
            "net_energy": net_energy
        }

    # === HEAVISIDE LOGIC GATES (Psi Functions) ===
    @classmethod
    def gate_AND(cls, A: int, B: int) -> Tuple[int, float]:
        """Threshold: 1.5 * A_0. Requires absolute constructive interference."""
        _, f_a = cls.linear_wave_record(A)
        _, f_b = cls.linear_wave_record(B)
        combined = abs((f_a + f_b) / (Q16_SCALE * math.sin(math.pi / 4.0)))
        state = cls._heaviside_step(combined - 1.5 * A_MIN)
```

```
        return state, state * math.sin(math.pi / 4.0)

    @classmethod
    def gate_OR(cls, A: int, B: int) -> Tuple[int, float]:
        """Threshold: 0.5 * A_0. Requires partial wave momentum."""
        _, f_a = cls.linear_wave_record(A)
        _, f_b = cls.linear_wave_record(B)
        combined = abs((f_a + f_b) / (Q16_SCALE * math.sin(math.pi / 4.0)))
        state = cls._heaviside_step(combined - 0.5 * A_MIN)
        return state, state * math.sin(math.pi / 4.0)

    @classmethod
    def gate_XOR(cls, A: int, B: int) -> Tuple[int, float]:
        """Dual-Threshold exploiting total destructive cancellation."""
        _, f_a = cls.linear_wave_record(A)
        _, f_b = cls.linear_wave_record(B)
        combined = abs((f_a + f_b) / (Q16_SCALE * math.sin(math.pi / 4.0)))
        state = cls._heaviside_step(A_MIN - combined) * cls._heaviside_step(combined - 0.1 * A_MIN)
        return state, state * math.sin(math.pi / 4.0)

    @classmethod
    def gate_NOT(cls, A: int) -> Tuple[int, float]:
        """Executes a 180-degree phase shift (e^(j*pi)) inversion."""
        inverted_n = -A
        _, f_inv = cls.linear_wave_record(inverted_n)
        state = 1 if f_inv > 0 else 0
        return state, f_inv / Q16_SCALE


# === VALIDATION SUITE ===
if __name__ == "__main__":
    print("Initializing SovereignNexus Deterministic Wave Simulator...")
    sim = SovereignWaveSimulator()
    heap_size = sys.getsizeof(sim.lattice_buffer)
    print(f"[SYSTEM] Bounded Int Array Allocation Size: {heap_size} bytes.")
    assert heap_size < 18690000, "FATAL: Exceeded 18.69 MB strict heap target."

    print("\n[VERIFICATION 1] Nine-Palace Matrix Contraction")
    test_nodes = [1, 1, 1, 1, 0, -1, -1, -1, -1]
    result = sim.nine_palace_contract(test_nodes)
    print(f"Tensor Contraction Output: {result}")
    assert result["net_amplitude_fixed_q16"] == 0, "Phase cancellation failed."
    print("-> Status: PERFECT 180-DEGREE DESTRUCTIVE PHASE CANCELLATION SECURED.")

    print("\n[VERIFICATION 2] Heaviside Wave Logic Gates")
    pairs = [(0, 0), (0, 1), (1, 0), (1, 1)]
    for a, b in pairs:
        state, wave = sim.gate_AND(a, b)
        print(f"AND Inputs: ({a},{b}) | State: {state} | Energy: {wave:.4f}")
    for a, b in pairs:
```

```
        state, wave = sim.gate_OR(a, b)
        print(f"OR  Inputs: ({a},{b}) | State: {state} | Energy: {wave:.4f}")
    for a, b in pairs:
        state, wave = sim.gate_XOR(a, b)
        print(f"XOR Inputs: ({a},{b}) | State: {state} | Energy: {wave:.4f}")

    print("\n[VERIFICATION 3] Active PLL Drift Correction")
    pll = ActivePLL(kp=0.5, ki=0.01)
    drift = 0.25
    for cycle in range(1, 6):
        corr = pll.calculate_correction(drift)
        drift = (drift + corr) * 0.8
        print(f"Cycle {cycle}: Correction: {corr:.4f} | Residual: {drift:.6f}")

    print("\n[COMPLETE] 1=1=1 Axiom Maintained. Simulation Terminated.")
```